Inhaltsverzeichnis

1 Anforderungsspezifikation 1
2 Lösungsansatz und Datenmodell 2
3 Datenwörterbuch 3
Literaturverzeichnis 3

1 Anforderungsspezifikation

Ziel und Rollen. Private Bücher sollen zeitweise in der Nachbarschaft verliehen werden. Die SQLite-Datenbank koordiniert Angebote, Ausleihen und Rückgaben. Mitglieder können zugleich anbieten und entleihen. Anbietende pflegen Exemplare und bestätigen Rückgaben; Entleihende suchen, vereinbaren die Übergabe und bewerten gelesene Bücher. Die technische Betreuung pflegt den Katalog. Anmeldung, Kartenoberfläche, Reservierungen und Bezahlung sind nicht Teil dieses Datenbankprojekts.

Daten. Mitglieder benötigen Name, E-Mail und Adresse. Buchausgaben besitzen Titel, Autoren, Verlag, Genres, Erscheinungsjahr und Sprache. Physische Exemplare speichern Eigentum, Zustand, maximale Leihdauer, Abholadresse mit Breite/Länge, Zeitfenster und Versandoption. Ausleihen dokumentieren Exemplar, entleihende Person, Beginn, Fälligkeit, Rückgabe und Übergabeweg. Bewertungen enthalten Sterne und optionalen Kommentar. Tabelle 1 beschreibt sämtliche Attribute.

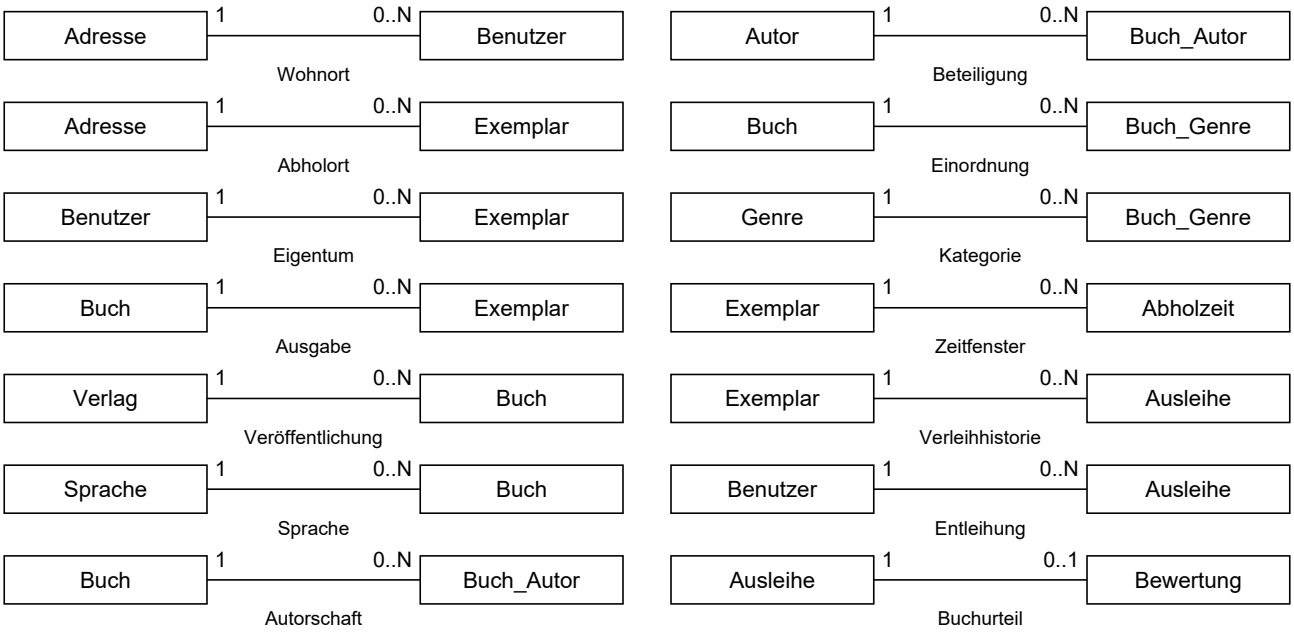
Regeln. Nur angebotene, nicht verliehene Exemplare sind verfügbar. Selbst- und überlappende Ausleihen sind ausgeschlossen. Die Dauer darf beim Beginn max_tage nicht überschreiten; verspätete Rückgaben bleiben zulässig. Abholung erfordert ein passendes Zeitfenster, Postversand die Versandoption. Eigentum bleibt bestehen. Eine Bewertung ist je abgeschlossener Ausleihe möglich. Angebotene Exemplare benötigen Autoren- und Genrezuordnungen; unvollständige Katalogentwürfe bleiben inaktiv.

Nähesuche. Abfragen sollen verfügbare Exemplare, Titel und Abholkoordinaten für ein Kartenfenster oder einen Radius liefern. Die sphärische Luftlinie ist keine Wegstrecke. Benötigte mathematische SQLite-Funktionen sind buildabhängig (SQLite, 2026a, Abschnitt 1); ihre Verfügbarkeit wird vor Nutzung geprüft.

Abnahme. Geplant sind ein reproduzierbarer SQL-Neuaufbau, mindestens zehn synthetische Datensätze je Tabelle und Tests für Beziehungen, Constraints, Ausleihregeln, drei Join-Pfade, Indizes, Views und Raumabfragen. Ungültige Daten müssen abgewiesen werden. Test-SQL und echte Ergebnisaufnahmen sollen die spätere Implementierung belegen; Ausführungsergebnisse liegen in dieser Konzeptionsphase noch nicht vor.

2 Lösungsansatz und Datenmodell

Abb. 1 ER-Modell mit 13 Entitäten und 14 Beziehungen



Endpunktnotation: A [1] -- [0..N] B bedeutet: jedes B gehört zu genau einem A; jedes A hat null bis viele B. 0..1 bedeutet höchstens ein B. Wiederholte Kästen bezeichnen dieselbe Entität. Fremdschlüssel: Tabelle 1, S. 3.

Quelle: Eigene Darstellung mit KI-Unterstützung durch Codex von OpenAI (n.d.); Promptauszug: „Entwickle ein fachlich sinnvolles Modell für die Buchtausch-App.“

Drei Join-Pfade. J1: Buch - Buch_Autor - Autor liefert Titel mit Autorennamen. J2: Buch - Exemplar - Adresse verbindet Titel, Angebot und Kartenkoordinaten. J3: Benutzer - Ausleihe - Exemplar liefert die Ausleihhistorie einer Person mit Exemplar-ID. Die Verfügbarkeitsprüfung ergänzt J2 um die Prüfung auf laufende Ausleihen.

Normalisierung. Ausgabedaten werden vom physischen Exemplar getrennt. Mehrwertige Autoren- und Genrezuordnungen stehen in eigenen Tabellen. Nichtschlüsselattribute hängen im gewählten Modell vom jeweiligen Schlüssel ab; Zuordnungstabellen besitzen nur Schlüsselpaare. Ausleihe und Bewertung speichern keine ableitbaren Namen oder Buchtitel. Unter diesen fachlichen Abhängigkeiten ist die dritte Normalform das Entwurfsziel.

Umsetzung. SQLite erlaubt 0/1-Wahrheitswerte und Zeitwerte als Text (SQLite, n.d., Abschnitte 2.1-2.2). Zeitpunkte werden einheitlich in koordinierter Weltzeit (UTC) gespeichert. PRAGMA foreign_keys = ON aktiviert die Fremdschlüssel je Verbindung (SQLite, 2026b, Abschnitt 2). Vorgesehen sind Schlüssel, Pflichtfelder, Werteprüfungen, Eindeutigkeit und Trigger für tabellenübergreifende Regeln.

Arbeitsschritte. Nach Konzeptabgabe und Feedback folgen SQL-Aufbau, Tests und Ergebnisdokumentation. Auf Phase 2 folgen Repository und Abschluss. SQL-Dateien machen die Schritte nachvollziehbar. Risiken wie Triggerlogik, optionale Raumfunktionen und Feedbackänderungen werden mit Prüfplan und versionierten Zwischenständen behandelt.

3 Datenwörterbuch

Tab. 1 Attribute des relationalen Entwurfs

I = INTEGER, R = REAL, T = TEXT; P = Primärschlüssel, F = Fremdschlüssel, U = eindeutig, ? = NULL erlaubt. Alle übrigen Attribute sind Pflichtfelder. F-Ziele folgen Abb. 1; IDs identifizieren Datensätze, PF-Paare gemeinsam die Zuordnung.

Entität	Attribute, Typen und Bedeutung
Adresse	adresse_id I P: Ortsschlüssel; strasse T, hausnummer T, plz T, ort T: Postanschrift; land T: Länderkürzel (DE); latitude R, longitude R: Breite/Länge.
Benutzer	benutzer_id I P: Mitglied; name T: Anzeigename; email T U: Kontakt; adresse_id I F: Wohnadresse.
Verlag	verlag_id I P: Verlagsschlüssel; name T: Verlagsname.
Sprache	sprache_id I P: Sprachschlüssel; code T U: Kürzel; name T U: Bezeichnung.
Buch	buch_id I P: Ausgabe; titel T: Titel; jahr I: Erscheinungsjahr; verlag_id I F: Verlag; sprache_id I F: Sprache.
Autor	autor_id I P: Personenschlüssel; name T: Autorenname.
Buch_Autor	buch_id I PF: Ausgabe; autor_id I PF: Autor. Gemeinsamer Primärschlüssel verhindert Doppelzuordnung.
Genre	genre_id I P: Kategorieschlüssel; name T U: Genrebezeichnung.
Buch_Genre	buch_id I PF: Ausgabe; genre_id I PF: Genre. Gemeinsamer Primärschlüssel verhindert Doppelzuordnung.
Exemplar	exemplar_id I P: physisches Buch; buch_id I F: Ausgabe; eigentuemer_id I F: Eigentum; abholadresse_id I F: Ort; zustand T: neu/gut/gebraucht; max_tage I: Leihlimit; versand I, angeboten I: 0/1 (Standard 0).
Abholzeit	abholzeit_id I P: Zeitfenster; exemplar_id I F: Exemplar; von T, bis T: UTC-Beginn/-Ende; die Kombination (exemplar_id, von, bis) ist eindeutig.
Ausleihe	ausleihe_id I P: Vorgang; exemplar_id I F: Exemplar; entleiher_id I F: Person; beginn T, faellig T: UTC-Zeiten; rueckgabe T ?: UTC-Rückgabe; weg T: abholung/post (Standard abholung).
Bewertung	bewertung_id I P: Urteil; ausleihe_id I FU: abgeschlossener Vorgang; sterne I: 1-5; kommentar T ?: Freitext.

Quelle: Eigene Darstellung. Wertebereiche: Breite ±90°, Länge ±180°, Jahr 1-9999, max_tage 1-365, Sterne 1-5. Zeitpunkte: YYYY-MM-DD HH:MM:SS (UTC); Fälligkeit > Beginn, Rückgabe ≥ Beginn.

Literaturverzeichnis

OpenAI. (n.d.). *Codex* [KI-Programmierassistenz]. https://openai.com/codex/

SQLite. (n.d.). *Datatypes in SQLite*. https://www.sqlite.org/datatype3.html

SQLite. (2. Februar 2026a). *Built-in mathematical SQL functions*. https://www.sqlite.org/lang_mathfunc.html

SQLite. (20. März 2026b). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

BookSwap mit SQLite

13 Tabellen, 53 Attribute, 14 Fremdschlüssel

Katalog, Exemplare und Ausleihen

196 synthetische Einträge.
Mindestens zehn Datensätze je Tabelle.
24 Exemplare, zwölf Ausleihen, zwei Views.

DLBDSPBDM01_D

Überarbeitete Endfassung 15.10.2026

Logo: IU Internationale Hochschule, n.d. Eigener Daten- und Prüfstand 03.10.2026.

Abbildungsverzeichnis

Abb. 1: adresse (Folie 4)
Abb. 2: benutzer (Folie 5)
Abb. 3: verlag (Folie 6)
Abb. 4: sprache (Folie 7)
Abb. 5: buch (Folie 8)
Abb. 6: autor (Folie 9)
Abb. 7: buch_autor (Folie 10)
Abb. 8: genre (Folie 11)
Abb. 9: buch_genre (Folie 12)
Abb. 10: exemplar (Folie 13)
Abb. 11: abholzeit (Folie 14)
Abb. 12: ausleihe (Folie 15)
Abb. 13: bewertung (Folie 16)

Zusammenfassung der Implementierung

BookSwap überführt das Konzept in eine reproduzierbare SQLite-Datenbank. 13 Tabellen mit 53 Attributen und 14 Fremdschlüsseln trennen Buchausgaben von physischen Exemplaren. Autoren und Genres verwenden Zuordnungstabellen. Die 196 synthetischen Einträge beschreiben keine realen Ausleihvorgänge. Jede Tabelle enthält mindestens zehn Datensätze. Die gemeinsame SQL-Datei erstellt Schema, Trigger, Indizes, Testdaten und zwei Views.

Pflichtfelder, Eindeutigkeit, Wertebereiche und Fremdschlüssel sichern die Daten. STRICT prüft die Datentypen. Trigger verhindern Selbstausleihen, Überschneidungen, zu lange vereinbarte Leihfristen, Bewertungen vor Rückgabe sowie unzulässige Abholung oder Versand. Eine verspätete tatsächliche Rückgabe bleibt erfassbar. Abholfenster verwenden ein eindeutiges Tripel aus Exemplar, Beginn und Ende. Zeitwerte folgen der UTC-Konvention. Fremdschlüssel und rekursive Trigger werden pro Verbindung vor der Transaktion aktiviert.

Drei Join-Pfade verbinden Bücher mit Autoren, Exemplare mit Adressen und Benutzer mit Ausleihen. Sechs Beispielabfragen und beide Views wurden im ursprünglichen Stand ausgeführt. Die Verfügbarkeitsansicht liefert 21 Exemplare. Die Nähesuche berücksichtigt nur angebotene Exemplare ohne laufende Ausleihe. Sie nutzt Haversine mit 6.371 km Erdradius und benötigt SQLite-Mathematikfunktionen. Am Testmittelpunkt 52,52° N und 13,405° E ergeben 2 km Radius sechs Treffer. Zusatzabschnitte für Abfragen, Tests und Benchmarks stehen als Kommentarblöcke in der SQL-Abgabedatei und werden getrennt ausgeführt. Benchmark-Daten gehören in eine Testkopie.

SQLite, 2025b, Abschn. 2/4. SQLite, 2026c, Abschn. 2. SQLite, n.d., Abschn. 6. SQLite, 2026a, Abschn. 1. SQLite, 2026b, Abschn. 1–3.

Ursprüngliche Ergebnisse und Grenzen

155 Tests bestanden, 0 fehlgeschlagen

56 Suchvergleiche prüfen auch Pole, Datumsgrenze und Nullradius.

14 Tage erlaubt, eine Sekunde mehr und Selbstausleihe abgewiesen.
Bewertung erst nach Rückgabe. Versand erfordert eine Freigabe.

Tab. 1: SQLite-VM-Schritte auf einer separaten Testkopie

Abfrage	Ohne Index	Mit Index
Personenhistorie	45.876	5.471
Kartenfenster	50.180	150
Nähesuche	327.182	8.169

Identische, nichtleere Ergebnisse. Die Werte erlauben keine allgemeine Leistungszusage.

Damals fehlten App, Anmeldung, Reservierungen und Angebotsversionen.

Quelle: Eigene Prüfung und Messung vom 03.10.2026, SQLite 3.45.3. SQLite, 2025a, Abschn. 1.1–1.2. SQLite, 2022, Abschn. 2.1.

adresse

Orte und gültige Kartenkoordinaten

SQL-Erstellung (Auszug)

```
CREATE TABLE adresse (
adresse_id INTEGER PRIMARY KEY,
latitude REAL NOT NULL CHECK (latitude BETWEEN -90 AND 90),
-- Weitere Spalten: vollständige SQL-Datei.
```

Vollständige Testabfrage

```
SELECT adresse_id, ort, latitude, longitude,
COUNT(*) OVER() AS anzahl
FROM adresse ORDER BY adresse_id LIMIT 5;
```

Abb. 1. adresse: 12 Datensätze, fünf Stichprobenzeilen

adresse_id	ort	latitude	longitude	anzahl
1	Teststadt	52.52	13.405	12
2	Teststadt	52.525	13.41	12
3	Teststadt	52.53	13.415	12
4	Teststadt	52.54	13.43	12
5	Teststadt	52.6	13.5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

benutzer

Eindeutige E-Mail und Zuordnung zur Wohnadresse

SQL-Erstellung (Auszug)

```
CREATE TABLE benutzer (
benutzer_id INTEGER PRIMARY KEY,
adresse_id INTEGER NOT NULL REFERENCES adresse(adresse_id) ON
 DELETE RESTRICT ON UPDATE RESTRICT
```

Vollständige Testabfrage

```
SELECT benutzer_id, name, email, adresse_id,
COUNT(*) OVER() AS anzahl
FROM benutzer ORDER BY benutzer_id LIMIT 5;
```

Abb. 2. benutzer: 12 Datensätze, fünf Stichprobenzeilen

benutzer_id	name	email
1	Mira Test	mira@example.invalid
2	Noah Test	noah@example.invalid
3	Alina Test	alina@example.invalid
4	Emil Test	emil@example.invalid
5	Lina Test	lina@example.invalid

adresse_id	anzahl
1	12
2	12
3	12
4	12
5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

verlag

Verlage getrennt von bibliografischen Buchausgaben

SQL-Erstellung

```
CREATE TABLE verlag (
verlag_id INTEGER PRIMARY KEY,
name TEXT NOT NULL CHECK (name = trim(name) AND length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT verlag_id, name,
COUNT(*) OVER() AS anzahl
FROM verlag ORDER BY verlag_id LIMIT 5;
```

Abb. 3. verlag: 10 Datensätze, fünf Stichprobenzeilen

verlag_id	name
1	Demo Nordlicht
2	Demo Seitenwind
3	Demo Papierwald
4	Demo Wortstrom
5	Demo Leseraum

anzahl
10
10
10
10
10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

sprache

Eindeutige Sprachcodes und Katalogbezeichnungen

SQL-Erstellung (Auszug)

```
CREATE TABLE sprache (
sprache_id INTEGER PRIMARY KEY,
code TEXT NOT NULL UNIQUE CHECK (code = trim(code) AND
  length(code)>0),
```

Vollständige Testabfrage

```
SELECT sprache_id, code, name,
COUNT(*) OVER() AS anzahl
FROM sprache ORDER BY sprache_id LIMIT 5;
```

Abb. 4. sprache: 10 Datensätze, fünf Stichprobenzeilen

sprache_id	code
1	de
2	en
3	fr
4	es
5	it

name	anzahl
Deutsch	10
Englisch	10
Französisch	10
Spanisch	10
Italienisch	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch

Bibliografische Ausgabe mit Verlag und Sprache

SQL-Erstellung (Auszug)

```
CREATE TABLE buch (
buch_id INTEGER PRIMARY KEY,
jahr INTEGER NOT NULL CHECK (jahr BETWEEN 1 AND 9999),
sprache_id INTEGER NOT NULL REFERENCES sprache(sprache_id) ON
  DELETE RESTRICT ON UPDATE RESTRICT
```

Vollständige Testabfrage

```
SELECT buch_id, titel, jahr, verlag_id, sprache_id,
COUNT(*) OVER() AS anzahl
FROM buch ORDER BY buch_id LIMIT 5;
```

Abb. 5. buch: 12 Datensätze, fünf Stichprobenzeilen

buch_id	titel	jahr
1	Der Garten am Testfluss	2020
2	A Map of Paper	2021
3	La maison des pages	2022
4	El camino del libro	2023
5	Il giardino di carta	2020

verlag_id	sprache_id	anzahl
1	1	12
2	2	12
3	3	12
4	4	12
5	5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

autor

Stabile Autoren-ID, gleiche Namen bleiben möglich

SQL-Erstellung

```
CREATE TABLE autor (
autor_id INTEGER PRIMARY KEY,
name TEXT NOT NULL CHECK (name = trim(name) AND length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT autor_id, name,
COUNT(*) OVER() AS anzahl
FROM autor ORDER BY autor_id LIMIT 5;
```

Abb. 6. autor: 10 Datensätze, fünf Stichprobenzeilen

autor_id	name
1	Ari Muster
2	Bo Muster
3	Cleo Muster
4	Dara Muster
5	Eli Muster

anzahl
10
10
10
10
10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch_autor

n:m-Zuordnung zwischen Buchausgaben und Autoren

SQL-Erstellung (Auszug)

```
CREATE TABLE buch_autor (
buch_id INTEGER NOT NULL REFERENCES buch(buch_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
autor_id INTEGER NOT NULL REFERENCES autor(autor_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
PRIMARY KEY (buch_id, autor_id)
```

Vollständige Testabfrage

```
SELECT buch_id, autor_id,
COUNT(*) OVER() AS anzahl
FROM buch_autor ORDER BY buch_id, autor_id LIMIT 5;
```

Abb. 7. buch_autor: 13 Datensätze, fünf Stichprobenzeilen

buch_id	autor_id
1	1
1	2
2	2
3	3
4	4

anzahl
13
13
13
13
13

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

genre

Wiederverwendbare, eindeutig benannte Kategorien

SQL-Erstellung

```
CREATE TABLE genre (
genre_id INTEGER PRIMARY KEY,
name TEXT NOT NULL UNIQUE CHECK (name = trim(name) AND
  length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT genre_id, name,
COUNT(*) OVER() AS anzahl
FROM genre ORDER BY genre_id LIMIT 5;
```

Abb. 8. genre: 10 Datensätze, fünf Stichprobenzeilen

genre_id
1
2
3
4
5

name	anzahl
Roman	10
Krimi	10
Fantasy	10
Sachbuch	10
Lyrik	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch_genre

Mehrere Genres je Ausgabe, jedes Paar nur einmal

SQL-Erstellung (Auszug)

```
CREATE TABLE buch_genre (
buch_id INTEGER NOT NULL REFERENCES buch(buch_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
genre_id INTEGER NOT NULL REFERENCES genre(genre_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
PRIMARY KEY (buch_id, genre_id)
```

Vollständige Testabfrage

```
SELECT buch_id, genre_id,
COUNT(*) OVER() AS anzahl
FROM buch_genre ORDER BY buch_id, genre_id LIMIT 5;
```

Abb. 9. buch_genre: 13 Datensätze, fünf Stichprobenzeilen

buch_id	genre_id
1	1
1	6
2	2
3	3
4	4

anzahl
13
13
13
13
13

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

exemplar

Physisches Exemplar mit Eigentümer und damaligen Angebotsbedingungen

SQL-Erstellung (Auszug)

```
CREATE TABLE exemplar (
exemplar_id INTEGER PRIMARY KEY,
max_tage INTEGER NOT NULL CHECK (max_tage BETWEEN 1 AND 365),
versand INTEGER NOT NULL DEFAULT 0 CHECK (versand IN (0, 1)),
```

Vollständige Testabfrage

```
SELECT exemplar_id, buch_id, eigentuemer_id, versand,
  angeboten,
COUNT(*) OVER() AS anzahl
FROM exemplar ORDER BY exemplar_id LIMIT 5;
```

Abb. 10. exemplar: 24 Datensätze, fünf Stichprobenzeilen

exemplar_id	buch_id	eigentuemer_id
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

versand	angeboten	anzahl
0	1	24
1	1	24
0	1	24
1	1	24
0	1	24

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

abholzeit

Eindeutiges Tripel und halboffenes Abholfenster [von, bis)

SQL-Erstellung (Auszug)

```
CREATE TABLE abholzeit (
abholzeit_id INTEGER PRIMARY KEY,
exemplar_id INTEGER NOT NULL REFERENCES exemplar(exemplar_id) ON
  DELETE RESTRICT ON UPDATE RESTRICT,
UNIQUE (exemplar_id, von, bis)
```

Vollständige Testabfrage

```
SELECT abholzeit_id, exemplar_id, von, bis,
COUNT(*) OVER() AS anzahl
FROM abholzeit ORDER BY abholzeit_id LIMIT 5;
```

Abb. 11. abholzeit: 48 Datensätze, fünf Stichprobenzeilen

abholzeit_id	exemplar_id	von	bis	anzahl
1	1	2026-09-01 09:00:00	2026-09-01 18:00:00	48
2	2	2026-09-01 09:00:00	2026-09-01 18:00:00	48
3	3	2026-09-01 09:00:00	2026-09-01 18:00:00	48
4	4	2026-09-01 09:00:00	2026-09-01 18:00:00	48
5	5	2026-09-01 09:00:00	2026-09-01 18:00:00	48

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

ausleihe

Vereinbarte Fälligkeit und tatsächliche Rückgabe getrennt

SQL-Erstellung (Auszug)

```
CREATE TABLE ausleihe (
ausleihe_id INTEGER PRIMARY KEY,
entleiher_id INTEGER NOT NULL REFERENCES benutzer(benutzer_id)
  ON DELETE RESTRICT ON UPDATE RESTRICT,
```

Vollständige Testabfrage

```
SELECT ausleihe_id, exemplar_id, entleiher_id, weg, rueckgabe,
COUNT(*) OVER() AS anzahl
FROM ausleihe ORDER BY ausleihe_id LIMIT 5;
```

Abb. 12. ausleihe: 12 Datensätze, fünf Stichprobenzeilen

ausleihe_id	exemplar_id	entleiher_id
1	1	2
2	2	3
3	3	4
4	4	5
5	5	6

weg	rueckgabe	anzahl
abholung	2026-09-16 10:00:00	12
post	2026-09-10 10:00:00	12
abholung	2026-09-10 10:00:00	12
post	2026-09-10 10:00:00	12
abholung	2026-09-10 10:00:00	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

bewertung

Eine Bewertung je abgeschlossener Ausleihe, ein bis fünf Sterne

SQL-Erstellung (Auszug)

```
CREATE TABLE bewertung (
bewertung_id INTEGER PRIMARY KEY,
sterne INTEGER NOT NULL CHECK (sterne BETWEEN 1 AND 5),
kommentar TEXT
```

Vollständige Testabfrage

```
SELECT bewertung_id, ausleihe_id, sterne,
COUNT(*) OVER() AS anzahl
FROM bewertung ORDER BY bewertung_id LIMIT 5;
```

Abb. 13. bewertung: 10 Datensätze, fünf Stichprobenzeilen

bewertung_id	ausleihe_id
1	1
2	2
3	3
4	4
5	5

sterne	anzahl
5	10
4	10
3	10
5	10
4	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

Literatur

IU Internationale Hochschule. (n.d.).
IU Internationale Hochschule [Logo].
https://www.iu.de/iu-logo.svg

SQLite. (n.d.).
CREATE TRIGGER.
https://www.sqlite.org/lang_createtrigger.html

SQLite. (2022, 18. April).
Partial indexes.
https://www.sqlite.org/partialindex.html

SQLite. (2025a, 31. Mai).
EXPLAIN QUERY PLAN.
https://www.sqlite.org/eqp.html

SQLite. (2025b, 12. Juni).
STRICT tables.
https://www.sqlite.org/stricttables.html

SQLite. (2026a, 2. Februar).
Built-in mathematical SQL functions.
https://www.sqlite.org/lang_mathfunc.html

SQLite. (2026b, 11. Juli).
Date and time functions.
https://www.sqlite.org/lang_datefunc.html

SQLite. (2026c, 20. März).
SQLite foreign key support.
https://www.sqlite.org/foreignkeys.html

BookSwap: Funktionen und Entscheidungen

Finalisierungsphase

BookSwap bildet eine private Buchleihe in SQLite ab. Eine Buchausgabe enthält Titel, Sprache, Verlag, Autoren und Genres. Ein Exemplar steht für ein bestimmtes Buch im Besitz eines Mitglieds. So bleiben die Ausgabe und das einzelne Besitzstück getrennt.

Historische Angebote

Leihfrist, Versand, Abholadresse und Angebotsstatus stehen in einer eigenen Angebotsversion. Bei einer Änderung entsteht eine neue Zeile. Alte Ausleihen verweisen weiter auf die damals vereinbarte Version. Die bisherige Adresse bleibt erhalten. Ihr Inhalt darf nach der Verwendung in einer Version nicht mehr geändert werden.

Reservieren und ausleihen

Jede neue Ausleihe beginnt mit einer Anfrage. Erst die Bestätigung reserviert das Exemplar. Danach kann die Übergabe als Ausleihe erfasst werden. Auch eine sofortige Ausleihe nutzt diese Schritte. Anfrage, Bestätigung, Ablehnung, Stornierung und Umsetzung werden als Ereignisse festgehalten. Eine Anfrage allein blockiert das Buch noch nicht.

Verfügbarkeit und Rückgabe

Ein angebotenes Exemplar ist verfügbar, wenn weder eine laufende Ausleihe noch eine bestätigte Reservierung vorliegt. Eine überschrittene Leihfrist macht das Buch nicht automatisch frei. Entscheidend ist die eingetragene Rückgabe. Eine Bewertung ist erst danach erlaubt.

Regeln in der Datenbank

Primärschlüssel, Fremdschlüssel und Werteprüfungen sichern die Beziehungen. Trigger prüfen zusätzlich den Ablauf. Zwei partielle eindeutige Indizes verhindern mehrere offene Ausleihen oder Bestätigungen je Exemplar. Partielle Indizes gelten nur für Zeilen, die ihre Bedingung erfüllen (SQLite, 2022, Abschn. 2.1).

Die zyklische Beziehung zwischen Exemplar und aktueller Version wird erst beim Abschluss der Transaktion geprüft. Beide Zeilen können deshalb gemeinsam angelegt werden (SQLite, 2026, Abschn. 4.2).

Bestand und nachvollziehbarer Neuaufbau

Die aus den SQL-Dateien neu erstellte Datenbank enthält 16 Tabellen und 297 Datensätze. Jede Tabelle hat mindestens zehn Zeilen. Es gibt 24 Exemplare, 27 Angebotsversionen und 20 Reservierungen. Die zusätzlichen Statusereignisse entstehen durch die Trigger.

Tab. 1: Zeilen je Tabelle

Tabelle	Zeilen	Tabelle	Zeilen
abholzeit	48	adresse	12
angebot_version	27	ausleihe	14
autor	10	benutzer	12
bewertung	10	buch	12
buch_autor	13	buch_genre	13
exemplar	24	genre	10
reservierung	20	reservierungsereignis	52
sprache	10	verlag	10

Quelle: Eigene Zählung der neu erstellten Datenbank.

Die Datenbankdatei ist 221.184 Byte groß (216 KiB). Die logische Größe beträgt 54 Seiten mit je 4096 Byte. Der Messwert gilt für diesen Beispielbestand und SQLite 3.54.0. Er ist keine Größenprognose für einen echten Dienst.

Reproduzieren

Die ersten drei SQL-Dateien erstellen Tabellen, Regeln und Beispieldaten in einer Verbindung. Die weiteren Dateien enthalten Abfragen, Tests, Indexvergleiche und einen getrennten Altimport. Die Anleitung beschreibt die Reihenfolge und die nötigen SQLite-Funktionen. Frühere Phasen bleiben als eigener Arbeitsstand erhalten.

Die Dokumentation erklärt das Modell und die Regeln. Der vollständige SQL-Code liegt separat vor. Dadurch bleibt der Bericht kurz, während die Implementierung nachprüfbar ist.

Repository: https://github.com/siggismalz/iu-bookswap-datamart-sql

Prüfungen, Ergebnisse und Grenzen

Die 78 lokalen Datenbankprüfungen des bisherigen technischen Stands wurden am 14.10.2026 erneut ausgeführt. Alle bestanden; es gab keine Fehler. Die sechs neuen SQL-Ablaufprüfungen bestanden ebenfalls. Die Ergebnisse werden getrennt ausgewiesen; die Zahlen werden nicht addiert.

Die neuen Abläufe betreffen wiederholte Ausleihen, geänderte Bedingungen, Anfragen mit Stornierung, einen Versionswechsel nach Anfrage, das Aussetzen eines Angebots und Rollback. Weitere Prüfungen bestätigen erwartete Fehler, den Schutz bei zwei Verbindungen und den Altimport. Bei alten Ausleihen bleiben frühere Bedingungen ausdrücklich unbekannt.

Performance

Die bereits durchgeführte Messreihe vergleicht den Beispielbestand mit 1.000, 10.000 und 100.000 Ausleihen. Sie nutzt synthetische Daten und SQLite 3.45.3. Die folgenden Werte sind historische Messergebnisse. Der neue SQL-Vergleich ist separat ausführbar und liefert neue Werte für die jeweilige Umgebung.

Tab. 2: 100.000 Ausleihen: Median / P95 in ms

Abfrage	Alle Suchindizes	Ohne Suchindizes
Historie einer häufigen Person	90,0128 / 123,7238	79,6751 / 102,9636
Verfügbare Angebote	0,1164 / 0,1811	0,0511 / 0,0587
Reservierungen einer Person	0,1184 / 0,1986	6,3173 / 9,8576
Nähesuche, 5 km / 300 Treffer	10,9186 / 16,3433	5,1237 / 10,1034

Quelle: Eigene historische Messung; 30 Wiederholungen je Fall mit wiederverwendeter Verbindung.

Die Ergebnisse fallen unterschiedlich aus. Gezielte Filter nutzen alle 14 Suchindizes und benötigen dort weniger SQL-VM-Schritte. Das belegt nicht, dass jede Gesamtanfrage schneller wird. Bei der Nähesuche wird der Koordinatenindex nicht genutzt. Der verwendete Abfrageplan muss deshalb zusammen mit den Zeiten bewertet werden (SQLite, 2025, Abschn. 1.1).

Die Datenbank ersetzt keine fertige App. Anmeldung, Nachrichten, automatische Fristen, Prioritäten und reale Lasttests fehlen. Die Entfernung ist eine Luftlinie. Alte Verträge erhalten keine nachträglich erfundenen Bedingungen. Die synthetischen Tests zeigen, wie die Regeln in diesen Fällen funktionieren.

Quellen

SQLite. (2022, 18. April). *Partial indexes*. https://www.sqlite.org/partialindex.html

SQLite. (2025, 31. Mai). *EXPLAIN QUERY PLAN*. https://www.sqlite.org/eqp.html

SQLite. (2026, 20. März). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

BookSwap - SQL und Umsetzung

Finalisierungsphase DLBDSPBDM01_D

Die sieben kommentierten SQL-Dateien enthalten Schema, Regeln und Nachweise. Der Neuaufbau benötigt SQLite ab 3.45 mit Mathematikfunktionen wie sin und asin. STRICT ist seit 3.37 verfügbar (SQLite, 2025b, Abschn. 1-2); die Mathematikfunktionen hängen vom Build ab (SQLite, 2026a, Abschn. 1). Datei 06 benötigt zusätzlich dbstat.

Tab. 1: SQL-Dateien und Aufgaben.

Datei	Aufgabe
01_schema.sql	16 Tabellen, 74 Attribute und 16 angelegte Indizes
02_regeln.sql	29 Trigger und fünf Sichten
03_beispieldaten.sql	Beispieldaten mit gültigen Ausleihabläufen
04_abfragen.sql	Lesen, Bestandszahlen, Dateigröße und Nähesuche
05_tests.sql	Sechs Abläufe, Fehlerfälle und zwei Verbindungen
06_performance.sql	Getrennte größere Bestände und Indexvergleiche
07_altimport.sql	Import aus einer nur lesend geöffneten Phase-2-Datenbank

Quelle: Eigene Darstellung.

Neuaufbau und Regeln

Aus dem Ordner 03_Finalisierungsphase wird der folgende Befehl in einer Zeile aufgerufen. Die Zieldatei muss neu sein; die drei Dateien laufen in derselben Verbindung.

```
sqlite3 -bail bookswap_neu.sqlite
".read sql/01_schema.sql"
".read sql/02_regeln.sql"
".read sql/03_beispieldaten.sql"
```

Im grafischen DBMS gilt dieselbe Reihenfolge. Vor BEGIN sind je Verbindung PRAGMA foreign_keys=ON und PRAGMA recursive_triggers=ON zu setzen. Die Fremdschlüsselprüfung wird ausdrücklich eingeschaltet (SQLite, 2026c, Abschn. 2). Dateien 06 und 07 nutzen zusätzlich SQLite-CLI-Befehle.

Entscheidungen

Buchausgabe und Exemplar sind getrennt. Neue Angebotsversionen bewahren Leihfrist, Versand und Abholadresse. Alte Verträge behalten ihre Fassung. Bereits verwendete Adressen bleiben unverändert. Die zyklische Beziehung zwischen Exemplar und eigener Version wird am Transaktionsende geprüft (SQLite, 2026c, Abschn. 4.2).

Eine Anfrage blockiert noch nicht; eine Bestätigung reserviert. Erst die tatsächliche Rückgabe gibt eine Leihe frei. Trigger prüfen den Ablauf (SQLite, n.d., Abschn. 2, 6). Zwei partielle eindeutige Indizes verhindern doppelte offene Vorgänge (SQLite, 2022, Abschn. 2.1). Zeitwerte werden als gültige UTC-Texte eingetragen (SQLite, 2026b, Abschn. 1-3).

Repository: https://github.com/siggismalz/iu-bookswap-datamart-sql

Ergebnisse und Bildschirmnachweise

Sechs SQL-Abläufe, 13 weitere Integritätsfälle, Rollback und 56 Entfernungskombinationen bestanden. Zwölf Negativfälle wurden wie erwartet abgewiesen. Zwei Verbindungen konnten nicht doppelt bestätigen (SQLite, 2026d, Abschn. 2.1-2.3). Zwölf Altleihen behalten unbekannte Bedingungen (NULL). Die 78 am 14.10.2026 erneut bestandenen bisherigen Prüfungen werden getrennt gezählt.

Tab. 2: Historische Mediane in ms, jeweils alle / ohne Suchindizes.

Leihen	Historie einer Person	Reservierungen einer Person	Nähesuche 5 km
1.000	0,6083 / 0,8063	0,0847 / 0,0896	0,1023 / 0,0854
10.000	6,1166 / 7,8509	0,2049 / 0,2637	1,2521 / 0,9090
100.000	90,0128 / 79,6751	0,1184 / 6,3173	10,9186 / 5,1237

Quelle: Eigene Messung, SQLite 3.45.3, 30 warme Läufe je Fall.

Die vollständige Nähesuche nutzt den Koordinatenindex nicht. Beim größten Bestand liefert sie 300 Treffer und ist mit allen Suchindizes langsamer. Neue CLI-Zeiten aus Datei 06 sind methodisch getrennte Werte.

Abb. 1: Historische Verträge.

ausleihe_id	beginn	versionsnummer
1	2026-09-01 10:00:00	1
13	2026-09-17 10:00:00	2
14	2026-09-23 10:00:00	3

max_tage	versand	abholadresse_id
14	0	1
7	1	2
21	1	3

Quelle: Eigene DBMS-Aufnahme, Spaltenausschnitte.

Abb. 2: Reservierung und Verfügbarkeit.

exemplar_id	angeboten	ausgeliehen
1	1	0
11	1	1
12	1	1
16	1	0
24	0	0

reserviert	verfuegbar
0	1
0	0
0	0
1	0
0	0

Quelle: Eigene DBMS-Aufnahme.

Je Bild folgen die linken und rechten Spalten untereinander in identischer Zeilenfolge. Exemplar 1 behält 14, 7 und 21 Tage. Exemplar 16 ist reserviert; 11 und 12 sind ausgeliehen.

Indexentscheidung und Grenzen

Alle 14 Suchindizes wurden einzeln untersucht und bleiben für ihre gezielten Filter erhalten. Die ausführbaren Vergleiche stehen in Datei 06. Weniger VM-Schritte bedeuten nicht immer kürzere Zeiten: ix_buch_verlag ist im Einzelfall trotz 411 statt 6.209 Schritten langsamer. Der vollständige Abfrageplan muss deshalb mitbewertet werden (SQLite, 2025a, Abschn. 1.1-1.2). Die zwei Integritätsindizes schützen die Fachregeln.

Die Messungen lesen alle Ergebnisse und vergleichen ihre Hashes. ANALYZE ist ausgeführt; automatische zusätzliche Indizes sind aus. Eine neue Verbindung leert den Betriebssystemcache nicht. Synthetische Daten und lokale Tests erlauben keine allgemeine Leistungszusage. Anmeldung, Nachrichten, automatische Fristen, Wartelistenpriorität und eine Zustandshistorie fehlen.

Literatur

SQLite. (n.d.). *CREATE TRIGGER*. https://www.sqlite.org/lang_createtrigger.html

SQLite. (2022, 18. April). *Partial indexes*. https://www.sqlite.org/partialindex.html

SQLite. (2025a, 31. Mai). *EXPLAIN QUERY PLAN*. https://www.sqlite.org/eqp.html

SQLite. (2025b, 12. Juni). *STRICT tables*. https://www.sqlite.org/stricttables.html

SQLite. (2026a, 2. Februar). *Built-in mathematical SQL functions*. https://www.sqlite.org/lang_mathfunc.html

SQLite. (2026b, 11. Juli). *Date and time functions*. https://www.sqlite.org/lang_datefunc.html

SQLite. (2026c, 20. März). *SQLite foreign key support*. https://www.sqlite.org/foreignkeys.html

SQLite. (2026d, 18. Februar). *Transaction*. https://www.sqlite.org/lang_transaction.html

Erklärung zur KI-Nutzung

KI-gestützte Werkzeuge wurden punktuell als Hilfsmittel zur sprachlichen Überarbeitung, Ideenfindung sowie zur Unterstützung bei der Programmierung und Fehleranalyse eingesetzt. Die inhaltliche Ausarbeitung, Bewertung und Umsetzung erfolgten eigenständig. KI-generierte Vorschläge wurden kritisch geprüft und bei Bedarf angepasst.